

# ROAMATE

## 웹 기반 실시간 AI 여행 메이트

통합 개발 기획서 및 작업지침서

Frontend 1 명 + Backend 2 명 협업 기준

## LIVE MATE

AI 실시간 여행 동반자

여행의 변화를 먼저 이해하고,  
지금 필요한 한 가지를 제안합니다

날씨, 혼잡도, 이동시간 등 실시간 상황을 분석해  
일정을 자동으로 조정하고, 숨은 명소까지 추천해주는  
나만의 여행 비서



### 실시간 상황 분석

날씨, 혼잡도, 교통, 운영시간 등  
실시간으로 분석



### AI 일정 재구성

변수가 발생해도 최적의 일정을  
즉시 다시 설계



### 숨은 명소 추천

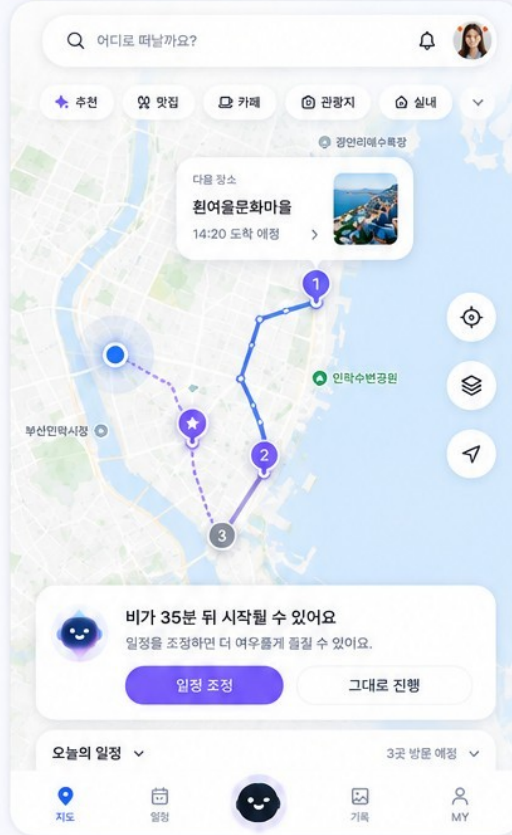
혼잡은 낮고 만족도는 높은  
히든 스팟을 경로에 자연스럽게



### 동행 맞춤 여행

동행자, 취향, 예산, 체력까지 고려한  
맞춤형 여행

## 1 지도 탭 현재 위치와 일정을 한눈에



### 상황 알림 카드

현재 상황과 다음 액션을  
간단히 안내



비가 35분 뒤 시작될 수 있어요

일정 조정

그대로 진행

### 다음 장소 카드

다음 장소의 도착 시간과  
사진을 미리 확인



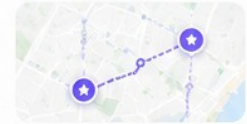
다음 장소

현여울문화마을

14:20 도착 예정

### 경로 미리보기

전체 동선과 히든 스팟을  
직관적으로 확인



## 2 일정 탭

시간 순으로 오늘 일정을 관리



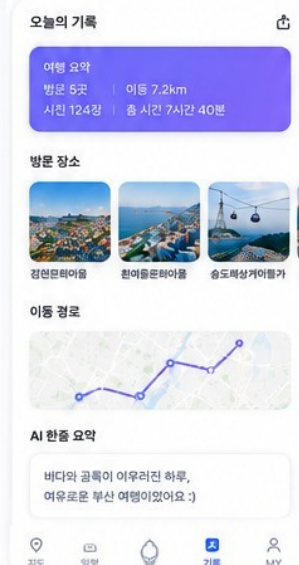
## 3 AI 비서 탭

AI와 대화하여 일정을 조정



## 4 기록 탭

여행 기록을 자동으로 정리



## 5 MY 탭

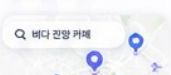
여행 설정과 개인 정보를 관리



## 주요 사용 흐름 4단계

### 1 탐색

가고 싶은 곳을 찾고  
일정을 만듭니다



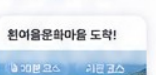
### 2 이동

길 안내를 받으며  
다음 장소로 이동해요



### 3 체류

도착하면 AI가 코스와  
정보를 안내해줘요



### 4 기록

여행이 끝나면 자동으로  
기록을 완성해줘요



## 핵심 차별점



실시간 상황 기반 자동 조정



숨은 명소를 자연스럽게 연결



대화보다 빠른 제안과 실행

## 문서 목적

화면 설계, 외부 API, 백엔드 역할 분담, REST API 계약, 데이터 모델, 핵심 코드, 통합 순서를 하나의 기준으로 고정한다. 각 담당자는 자신의 절을 순서대로 구현하고, 공통 API 계약에 맞춰 통합한다.

# 목차

| 구분 | 내용                      |
|----|-------------------------|
| 1  | 개발 목표와 완성 범위            |
| 2  | 웹 화면 구조와 사용자 흐름         |
| 3  | 전체 시스템 아키텍처             |
| 4  | 역할 분담과 협업 규칙            |
| 5  | 외부 API 선정 및 적용 방식       |
| 6  | 공통 REST API 계약          |
| 7  | Backend A: 관광 데이터·지도·날씨 |
| 8  | Backend B: 여행·일정·AI     |
| 9  | Frontend: 웹 화면·지도·상태관리  |
| 10 | 데이터베이스 설계               |
| 11 | 핵심 기능별 구현 절차            |
| 12 | 통합·테스트·완료 기준            |
| 부록 | 환경변수·실행 명령·출처           |

## 구현 우선순위

1 차 완성은 지도, 관광지 검색, 일정, 날씨 기반 영향 분석, AI 재설계, 사용자 승인·적용이다. 기록·즐거찾기·도착 감지·히든 스팟은 2 차 기능으로 연결하되 API 와 데이터 모델은 본 문서에 포함한다.

# 1. 개발 목표와 완성 범위

## 1.1 최종 서비스 정의

ROAMATE는 웹 브라우저에서 실행되는 지도 중심 여행 서비스다. 사용자의 현재 위치, 일정, 날씨, 운영시간, 이동시간과 상태를 분석해 여행에 영향을 주는 변수를 찾고, 한국관광공사 관광정보 안에서 대체 장소와 새로운 일정을 제안한다.

### 핵심 원칙

AI가 장소·날씨·운영시간·이동시간을 임의로 만들지 않는다. 외부 API와 서버 규칙으로 검증한 후보만 AI에 전달하며, 사용자가 승인하기 전에는 기존 일정을 변경하지 않는다.

## 1.2 MVP 필수 기능

| 영역     | 필수 기능                     | 완료 조건                 |
|--------|---------------------------|-----------------------|
| 지도     | 네이버 지도, 현재 위치, 일정 마커, 경로선 | 지도 위에 현재 일정과 경로 표시    |
| 관광정보   | 검색, 주변 장소, 상세, 이미지        | 한국관광공사 데이터가 카드·마커로 출력 |
| 일정     | 생성, 조회, 수정, 삭제, 순서 변경     | 시간순 타임라인과 DB가 동기화     |
| 실시간 상황 | 날씨, 사용자 상태, 운영시간 충돌       | 영향 일정과 위험 등급 반환       |
| AI 재설계 | 대체 후보 선정, 일정 생성, 검증       | 변경 전후 비교 결과 반환        |
| 적용     | 사용자 승인·거절, 변경 이력          | 승인 시 DB·지도·일정 갱신      |

## 1.3 2차 기능

- 도착 감지와 현장 가이드
- 히든 스팟 추천
- 여행 기록 자동 생성
- 즐겨찾기와 MY 설정
- 사진 업로드·AI 여행 요약

## 2. 웹 화면 구조와 사용자 흐름

### 2.1 페이지 구성

| Route        | 화면    | 주요 컴포넌트                                       | 주요 API                  |
|--------------|-------|-----------------------------------------------|-------------------------|
| /            | 지도 홈  | SearchBar, NaverMap, AlertCard, AssistantDock | places, trip, weather   |
| /trips/:id   | 일정    | Timeline, ScheduleCard, RouteSummary          | trip, schedules, route  |
| /assistant   | AI 비서 | ChatPanel, QuickActions, ActionResult         | assistant/chat, replans |
| /records/:id | 여행 기록 | VisitGallery, RouteHistory, AiSummary         | records, visits         |
| /my          | MY    | PreferenceForm, SavedPlaces, TripList         | users/me, preferences   |

### 2.2 핵심 사용 흐름

1. 사용자가 지도에서 관광지를 검색하거나 주변 장소를 선택한다.
2. 선택한 장소를 일정에 추가하고 순서를 정한다.
3. 현재 위치와 사용자 상태를 서버에 전달한다.
4. 서버가 날씨·운영시간·경로를 조회해 일정 영향을 분석한다.
5. 재설계가 필요하면 검증된 대체 후보를 만들고 AI가 변경안을 작성한다.
6. 프론트가 변경 전후를 비교해 보여준다.
7. 사용자가 적용하면 DB 일정과 지도 경로를 갱신한다.

### 2.3 화면 상태

| 상태    | UI 동작                 | 서버 동작               |
|-------|-----------------------|---------------------|
| 탐색    | 검색·카테고리·관광지 카드 강조     | 관광지 검색/주변 조회        |
| 이동    | 검색 UI 축소, 다음 장소·경로 강조 | Directions 조회       |
| 체류    | 장소 상세·체류 가이드 표시       | 도착 상태 저장            |
| 변수 대응 | 경고 대신 단일 결정 카드 표시     | Impact 분석·Replan 생성 |
| 기록    | 방문 장소·이동 경로·사진 정리     | Visit·Record 저장     |

### 3. 전체 시스템 아키텍처

```

[Browser / React]
- NAVER Dynamic Map
- Search / Timeline / AI Bottom Sheet
  | HTTPS JSON
  v
[Spring Boot API Server]
- Place / Route / Weather
- Trip / Schedule / Context / Impact
- Replan / Assistant / Record
  |
  +--> 한국관광공사 OpenAPI
  +--> NAVER Directions / Reverse Geocoding
  +--> Weather API
  +--> Spring AI / LLM
  +--> MySQL
  
```

#### 3.1 데이터 흐름

| 단계 | 입력              | 처리                   | 출력              |
|----|-----------------|----------------------|-----------------|
| 1  | 위치·현재시각·상태      | Context 저장           | contextId       |
| 2  | contextId·남은 일정 | 날씨·운영시간·사용자 상태 분석    | ImpactResult    |
| 3  | 영향 일정·현재 위치     | 주변 관광지 조회·경로 계산·점수화  | 후보 10 개         |
| 4  | 기존 일정·후보·예약     | Spring AI 구조화 출력     | AiReplanResult  |
| 5  | AI 결과           | contentId·시간·운영시간 검증 | ReplanResponse  |
| 6  | 사용자 승인          | 트랜잭션으로 일정 교체         | 최신 TripResponse |

#### 3.2 구현 원칙

- 프론트는 화면과 사용자 입력만 담당하고 외부 API Secret 을 보관하지 않는다.
- Backend A 는 외부 데이터와 경로를 정규화한다.
- Backend B 는 여행 도메인과 AI 재설계를 담당한다.
- 백엔드 간 연결은 Controller 호출이 아니라 Service 인터페이스로 처리한다.
- 외부 API 실패 시 기존 일정을 유지하고 표준 오류를 반환한다.

## 4. 역할 분담과 협업 규칙

| 담당        | 책임                      | 브랜치                      | 완료 산출물                                 |
|-----------|-------------------------|--------------------------|----------------------------------------|
| Frontend  | 지도·UI/UX·상태관리·API 연결    | feature/frontend         | 웹 화면, API 연결, 반응형 UI                   |
| Backend A | 관광정보·지도·경로·날씨           | feature/backend-data-map | 외부 API Client, Place/Route/Weather API |
| Backend B | 여행·일정·Context·AI·Replan | feature/backend-trip-ai  | 도메인 API, AI 재설계, 적용 로직                 |

### 4.1 파일 소유권

| 담당        | 수정 가능 패키지                                                  | 공동 수정 파일                        |
|-----------|------------------------------------------------------------|---------------------------------|
| Backend A | place, route, weather, location, external                  | application.yml.example, API.md |
| Backend B | trip, schedule, context, impact, replan, assistant, record | application.yml.example, DB.md  |
| Frontend  | frontend/src 전체                                            | README.md, API types            |

### 4.2 Git 규칙

```
git checkout develop
git pull origin develop
git checkout -b feature/backend-data-map
# 작업
git add .
git commit -m "feat: 관광공사 주변 관광지 API 연동"
git push origin feature/backend-data-map
```

- main: 제출·배포 가능한 버전만 유지
- develop: 통합 기준 브랜치
- 기능 브랜치는 작업자별로 분리
- API DTO 변경 시 프론트와 두 백엔드 모두에게 즉시 공유
- PR에는 요청/응답 예시와 Postman 캡처를 포함



## 5. 외부 API 선정 및 적용 방식

| 서비스                          | 사용 기능                         | 호출 위치     | 비고                  |
|------------------------------|-------------------------------|-----------|---------------------|
| NAVER Maps JavaScript API v3 | Dynamic Map, Marker, Polyline | Frontend  | ncpKeyId 사용         |
| NAVER Directions 5           | 자동차 경로·거리·시간                  | Backend A | 경유지 최대 5 개, 자동차 전용  |
| NAVER Reverse Geocoding      | 좌표→주소                         | Backend A | 현재 지역 표시            |
| 한국관광공사 국문 관광정보               | 검색·주변·상세·이미지                  | Backend A | contentId 를 기준으로 사용 |
| Weather API                  | 현재 날씨·예보                      | Backend A | 강수·기온·풍속 판정         |
| Spring AI                    | 의도 분석·일정 재설계                  | Backend B | 구조화 출력 + 서버 검증      |

### 5.1 한국관광공사 사용 오퍼레이션

| 목적     | 오퍼레이션              | 필수 파라미터                 | 저장 필드                                |
|--------|--------------------|-------------------------|--------------------------------------|
| 키워드 검색 | searchKeyword2     | keyword, pageNo         | contentid,title,mapx,mapy,firstimage |
| 위치 기반  | locationBasedList2 | mapX,mapY,radius        | contentid,title,dist                 |
| 공통 상세  | detailCommon2      | contentId               | overview,addr1,homepage              |
| 소개 상세  | detailIntro2       | contentId,contentTypeId | usetime,restdate,parking             |
| 이미지    | detailImage2       | contentId               | originimgurl,smallimageurl           |

### 5.2 제한사항

- Directions 5 는 자동차 경로만 제공하므로 웹 화면에 도보·대중교통 실시간 경로로 표시하지 않는다.
- 한국관광공사 데이터의 운영시간 누락 가능성을 고려해 정보 없음 상태를 지원한다.
- AI 는 이동시간과 운영시간을 계산하지 않는다.
- API 키는 .env 또는 서버 환경변수로만 관리한다.

## 6. 공통 REST API 계약

| 영역  | Method                | Endpoint                               | 담당 |
|-----|-----------------------|----------------------------------------|----|
| 관광지 | GET                   | /api/places                            | A  |
| 관광지 | GET                   | /api/places/nearby                     | A  |
| 관광지 | GET                   | /api/places/{contentId}                | A  |
| 경로  | POST                  | /api/routes/driving                    | A  |
| 위치  | GET                   | /api/locations/reverse-geocode         | A  |
| 날씨  | GET                   | /api/weather/current                   | A  |
| 여행  | POST/GET/PATCH/DELETE | /api/trips                             | B  |
| 일정  | POST/PATCH/DELETE     | /api/trips/{tripId}/schedules          | B  |
| 상황  | POST                  | /api/trips/{tripId}/context            | B  |
| 영향  | POST                  | /api/trips/{tripId}/impact/analyze     | B  |
| 재설계 | POST                  | /api/trips/{tripId}/replans            | B  |
| 적용  | POST                  | /api/trips/{tripId}/replans/{id}/apply | B  |
| AI  | POST                  | /api/assistant/chat                    | B  |
| 기록  | GET/POST              | /api/trips/{tripId}/record             | B  |

### 6.1 표준 오류 응답

```
{
  "code": "EXTERNAL_API_ERROR",
  "message": "관광정보를 불러오지 못했습니다.",
  "timestamp": "2026-07-20T14:20:00"
}
```

| HTTP | 용도            |
|------|---------------|
| 200  | 조회·수정 성공      |
| 201  | 생성 성공         |
| 204  | 삭제 성공         |
| 400  | 요청값 검증 실패     |
| 404  | 데이터 없음        |
| 409  | 현재 상태에서 처리 불가 |
| 502  | 외부 API 오류     |
| 500  | 서버 내부 오류      |



## 7. Backend A 작업지침: 관광 데이터·지도·날씨

### 7.1 패키지 구조

```
com.roadmate
├── place/controller, service, dto, entity, repository
├── route/controller, service, dto
├── weather/controller, service, dto
├── location/controller, service, dto
├── external
│   ├── kto
│   ├── naver
│   └── weather
```

### 7.2 PlaceResponse

```
public record PlaceResponse(
    String contentId,
    String contentTypeId,
    String title,
    String address,
    double latitude,
    double longitude,
    String imageUrl,
    boolean indoor,
    String openingHours,
    String closedDay,
    String source
) {}
```

### 7.3 관광지 검색 Controller

```
@RestController
@RequestMapping("/api/places")
@RequiredArgsConstructor
public class PlaceController {
    private final PlaceService placeService;

    @GetMapping
    public List<PlaceResponse> search(
        @RequestParam(required=false) String keyword,
        @RequestParam(required=false) String areaCode,
        @RequestParam(defaultValue="1") int page) {
        return placeService.search(keyword, areaCode, page);
    }

    @GetMapping("/nearby")
    public List<PlaceResponse> nearby(
        @RequestParam double latitude,
        @RequestParam double longitude,
        @RequestParam(defaultValue="5000") int radius,
        @RequestParam(required=false) Boolean indoor) {
        return placeService.searchNearby(latitude, longitude, radius, indoor);
    }
}
```

## 7. Backend A 작업지침: 핵심 외부 API 코드

### 7.4 관광공사 Client 처리 규칙

- mapX 는 longitude, mapY 는 latitude 로 변환
- 좌표가 없는 항목 제외
- 대표 이미지가 없으면 null
- 상세정보는 최초 조회 시 DB 캐시
- 외부 API 응답을 그대로 프론트에 전달하지 않음

```
public List<PlaceResponse> searchNearby(
    double latitude, double longitude, int radius) {
    URI uri = UriComponentsBuilder
        .fromHttpUrl(baseUrl + "/locationBasedList2")
        .queryParams("serviceKey", serviceKey)
        .queryParams("MobileOS", "ETC")
        .queryParams("MobileApp", "ROAMATE")
        .queryParams("_type", "json")
        .queryParams("mapX", longitude)
        .queryParams("mapY", latitude)
        .queryParams("radius", radius)
        .queryParams("numOfRows", 30)
        .build(true).toUri();

    JsonNode root = restClient.get().uri(uri)
        .retrieve().body(JsonNode.class);
    return mapper.toPlaces(root);
}
```

### 7.5 Directions API

```
public record Coordinate(double latitude, double longitude) {
    public String toNaverValue() {
        return longitude + "," + latitude;
    }
}

public RouteResponse calculate(RouteRequest request) {
    if (request.waypoints().size() > 5) {
        throw new BadRequestException("경유지는 최대 5 개입니다.");
    }
    NaverDirectionsResponse raw = client.getDrivingRoute(request);
    return routeMapper.toResponse(raw);
}
```

### 7.6 날씨 위험 판정

```
public boolean isOutdoorRisk(WeatherSnapshot w) {
    return Set.of("RAIN", "THUNDERSTORM", "SNOW")
        .contains(w.condition())
        || w.rainProbability() >= 70
        || w.temperature() >= 32
        || w.windSpeed() >= 10;
}
```

## 8. Backend B 작업지침: 여행·일정·AI

### 8.1 패키지 구조

```
com.roadmate
├── trip/controller, service, repository, dto, entity
├── schedule/controller, service, repository, dto, entity
├── context
├── impact
├── replan
├── assistant
└── record
```

### 8.2 여행 조회 응답

```
public record TripResponse(
    Long tripId,
    String title,
    String status,
    Coordinate currentLocation,
    List<ScheduleResponse> schedules,
    RouteResponse route
) {}
```

### 8.3 Context API

```
public record ContextRequest(
    double latitude,
    double longitude,
    LocalDateTime currentTime,
    Long currentScheduleId,
    UserCondition userCondition
) {}

@PostMapping("/{tripId}/context")
public ContextResponse createContext(
    @PathVariable Long tripId,
    @Valid @RequestBody ContextRequest request) {
    return contextService.create(tripId, request);
}
```

### 8.4 영향 분석 순서

8. 남은 일정 조회
9. WeatherQueryService 호출
10. 야외·해안·운영시간 충돌 판정
11. 사용자 상태 판정
12. 영향 일정과 severity 반환

## 8. Backend B 작업지침: AI 재설계

### 8.5 AI 출력 DTO

```
public record AiReplanResult(
    String summary,
    List<AiScheduleItem> schedules,
    List<AiChangeItem> changes
) {}

public record AiScheduleItem(
    String contentId,
    int orderNumber,
    LocalDateTime startTime,
    LocalDateTime endTime,
    String reason
) {}
```

### 8.6 Spring AI 호출

```
AiReplanResult result = chatClient.prompt()
    .system("제공된 관광 후보 안에서만 일정을 재설계한다.")
    .user(promptJson)
    .call()
    .entity(AiReplanResult.class);
```

### 8.7 서버 검증

```
public void validate(
    AiReplanResult result,
    Set<String> allowedContentIds,
    Set<String> mustVisitIds) {
    for (AiScheduleItem item : result.schedules()) {
        if (!allowedContentIds.contains(item.contentId())) {
            throw new InvalidAiResponseException("허용되지 않은 장소");
        }
    }
    validateTimeOverlap(result.schedules());
    validateMustVisit(result.schedules(), mustVisitIds);
    validateOpeningHours(result.schedules());
}
```

### 8.8 실패 대응

| 상황              | 처리               |
|-----------------|------------------|
| 구조화 출력 파싱 실패    | 동일 요청 1 회 재시도    |
| contentId 검증 실패 | 재시도 후 규칙 기반 일정   |
| 외부 API 실패       | 기존 일정 유지, 502 응답 |
| 적용 중 오류         | 트랜잭션 롤백          |

## 9. Frontend 작업지침: 웹 화면·지도·상태관리

### 9.1 기술 구조

```
frontend/src
├── api
├── components/map, assistant, itinerary, record, common
├── pages
├── hooks
├── store
├── types
└── styles
```

### 9.2 권장 라이브러리

| 목적       | 라이브러리                        |
|----------|------------------------------|
| 빌드       | Vite + React + TypeScript    |
| 서버 상태    | TanStack Query               |
| 전역 UI 상태 | Zustand                      |
| HTTP     | Axios                        |
| 라우팅      | React Router                 |
| 아이콘      | Lucide React                 |
| 지도       | NAVER Maps JavaScript API v3 |

### 9.3 API 타입 고정

```
export interface ReplanResponse {
  replanId: number;
  summary: string;
  before: TripMetrics;
  after: TripMetrics;
  changes: ReplanChange[];
  proposedSchedules: Schedule[];
  proposedRoute: RouteResponse;
}
```

### 9.4 상태 관리

| 상태                                       | 보관 위치                  |
|------------------------------------------|------------------------|
| trip, places, weather, replan            | TanStack Query         |
| selectedPlace, assistantSheet, activeTab | Zustand 또는 local state |
| API Secret                               | 프론트 저장 금지              |
| 지도 인스턴스·마커·Polyline                      | useRef                 |



## 9. Frontend 작업지침: 핵심 코드

### 9.5 네이버 지도 로더

```
let sdkPromise: Promise<void> | null = null;

export function loadNaverMapSdk() {
  if (window.naver?.maps) return Promise.resolve();
  if (sdkPromise) return sdkPromise;
  sdkPromise = new Promise((resolve, reject) => {
    const script = document.createElement("script");
    script.src = "https://oapi.map.naver.com/openapi/v3/maps.js"
      + `?ncpKeyId=${import.meta.env.VITE_NAVER_MAP_CLIENT_ID}`;
    script.onload = () => resolve();
    script.onerror = reject;
    document.head.appendChild(script);
  });
  return sdkPromise;
}
```

### 9.6 현재 위치와 도착 판정

```
navigator.geolocation.watchPosition(({ coords }) => {
  setLocation({ latitude: coords.latitude, longitude: coords.longitude });
});

const arrived = calculateDistanceMeters(
  current.latitude, current.longitude,
  target.latitude, target.longitude
) <= 100;
```

### 9.7 재설계 실행

```
const context = await createContext(tripId, {
  ...location,
  currentTime: new Date().toISOString(),
  userCondition
});
const impact = await analyzeImpact(tripId, context.contextId);
if (impact.replanRequired) {
  const proposal = await createReplan(tripId, {
    contextId: context.contextId,
    options
  });
  setReplan(proposal);
}
```

## 10. 데이터베이스 설계

| 테이블              | 핵심 필드                                  | 역할     |
|------------------|----------------------------------------|--------|
| users            | id,email,nickname                      | 사용자    |
| user_preferences | walking_level,prefer_indoor,theme      | 여행 선호  |
| trips            | title,start_date,end_date,status       | 여행 단위  |
| places           | content_id,title,lat,lng,opening_hours | 관광지 캐시 |
| schedules        | trip_id,place_id,order,start,end       | 일정     |
| travel_contexts  | trip_id,location,condition,weather     | 상황 스냅샷 |
| replans          | trip_id,context_id,status,summary      | 변경안    |
| replan_items     | replan_id,action,place_id,reason       | 변경 항목  |
| visits           | schedule_id,arrived_at,left_at         | 실제 방문  |
| saved_places     | user_id,place_id                       | 즐거찾기   |
| trip_records     | trip_id,summary,total_distance         | 여행 기록  |

### 10.1 주요 관계

```

USER 1 --- N TRIP
USER 1 --- 1 USER_PREFERENCE
TRIP 1 --- N SCHEDULE
PLACE 1 --- N SCHEDULE
TRIP 1 --- N TRAVEL_CONTEXT
TRIP 1 --- N REPLAN
REPLAN 1 --- N REPLAN_ITEM
SCHEDULE 1 --- 0..1 VISIT

```

### 10.2 저장 원칙

- 관광지 외부 데이터는 contentId 기준으로 upsert
- AI 변경안은 PENDING 상태로 저장 후 승인 시 적용
- 기존 일정은 적용 트랜잭션 안에서 백업·교체
- 위치 기록은 과도한 저장을 피하고 이벤트 중심으로 저장
- 사진은 파일 저장소 URL 만 DB 에 저장

## 11. 핵심 기능별 구현 절차

| 기능     | Frontend            | Backend A          | Backend B        |
|--------|---------------------|--------------------|------------------|
| 관광지 검색 | 키워드 입력·카드·마커        | KTO 검색·DTO 변환      | -                |
| 일정 생성  | 선택·순서 UI            | 경로 계산              | Trip/Schedule 저장 |
| 날씨 알림  | AlertCard           | Weather 조회         | Impact 판정        |
| AI 재설계 | Quick Action·비교 시트  | 후보·이동시간 제공         | AI 생성·검증·저장      |
| 적용     | 확인 버튼·지도 갱신         | 새 경로 계산            | 트랜잭션 적용          |
| 도착 감지  | watchPosition·거리 판정 | Reverse Geocode 선택 | Visit 저장         |
| 히든 스팟  | 경로 위 추천 카드          | 후보 점수 산정           | 일정 삽입 검증         |
| 여행 기록  | 갤러리·경로·요약           | 경로 데이터 제공          | Record 집계·AI 요약  |

### 11.1 히든 스팟 점수

```
score = 100
- detourMinutes * 2
+ (indoorMatch ? 15 : 0)
+ (themeMatch ? 20 : 0)
+ (dataCompleteness * 10)
+ (lowDemand ? 20 : 0)
- (closed ? 100 : 0);
```

### 11.2 도착 감지 기준

| 조건    | 기준                      |
|-------|-------------------------|
| 도착    | 목적지 100m 이내가 20 초 이상 유지 |
| 체류 시작 | 도착 후 사용자가 확인하거나 2 분 유지  |
| 이탈    | 300m 이상 벗어남             |
| 중복 방지 | 동일 일정 도착 이벤트 1 회 저장     |

## 12. 통합·테스트·완료 기준

### 12.1 7 일 일정

| 일차 | Frontend | Backend A     | Backend B          | 통합 결과     |
|----|----------|---------------|--------------------|-----------|
| 1  | 프로젝트·지도  | 키 발급·외부 호출    | Spring·DB·엔티티      | 지도/health |
| 2  | 검색·마커    | 관광지 API       | Trip/Schedule CRUD | 관광지·일정 출력 |
| 3  | 타임라인·경로  | Directions·날씨 | Context·Impact     | 경로·알림     |
| 4  | AI Dock  | 후보 점수         | Spring AI·검증       | 변경안 출력    |
| 5  | 비교·적용 UI | 경로 비교         | Replan 적용          | 일정 갱신     |
| 6  | 도착·기록 UI | 주소·히든 후보      | Visit·Record       | 확장 기능     |
| 7  | 반응형·QA   | Postman·오류    | 테스트·문서             | 최종 시연     |

### 12.2 핵심 테스트

| ID | 시나리오      | 성공 기준              |
|----|-----------|--------------------|
| T1 | 관광지 검색    | 검색 결과와 마커가 동일      |
| T2 | 경로 계산     | 거리·시간·Polyline 출력  |
| T3 | 비 + 야외 일정 | HIGH 영향 반환         |
| T4 | AI 재설계    | 허용 후보만 포함          |
| T5 | 필수 장소 유지  | mustVisit 누락 없음    |
| T6 | 적용        | DB 와 지도·타임라인 동시 갱신 |
| T7 | 외부 API 실패 | 기존 일정 유지·표준 오류     |
| T8 | 도착 감지     | 이벤트 1 회 저장         |

### 12.3 최종 완료 판정

- 부산 여행과 일정 3 곳을 생성한다.
- 네이버 지도에 마커와 자동차 경로를 표시한다.
- 현재 위치와 TIRED 상태를 저장한다.
- 비 예보로 야외 일정이 영향받다고 판정한다.
- 주변 실내 관광지와 이동시간을 계산한다.
- AI 가 변경안을 생성하고 서버 검증을 통과한다.
- 사용자가 적용하면 일정과 경로가 갱신된다.
- 변경 이력과 여행 기록이 조회된다.

## 부록 A. 환경변수와 실행 명령

### A.1 Frontend .env

```
VITE_API_BASE_URL=http://localhost:8080  
VITE_NAVER_MAP_CLIENT_ID=발급받은_ncpKeyId
```

### A.2 Backend 환경변수

```
DB_PASSWORD=...  
NAVER_MAP_CLIENT_ID=...  
NAVER_MAP_CLIENT_SECRET=...  
KTO_SERVICE_KEY=...  
WEATHER_API_KEY=...  
OPENAI_API_KEY=...
```

### A.3 실행

```
# backend  
cd backend  
./gradlew bootRun  
  
# frontend  
cd frontend  
npm install  
npm run dev
```

### A.4 기본 디렉터리

```
ROAMATE/  
├── frontend/  
├── backend/  
├── docs/API.md  
├── docs/DB.md  
├── docs/TEST.md  
└── README.md
```

## 부록 B. 공식 기술 근거 및 확인사항

API 및 라이브러리 사양은 개발 시작 시점에 공식 문서에서 다시 확인한다. 특히 한국관광공사 OpenAPI 는 2026 년 입출력 항목 변경 공지가 있었으므로 발급받은 활용신청 명세를 최종 기준으로 사용한다.

| 항목                           | 공식 근거                                                        |
|------------------------------|--------------------------------------------------------------|
| NAVER Maps JavaScript API v3 | ncpKeyId 로 웹 지도 SDK 로드                                       |
| NAVER Maps REST API          | Directions 5, Directions 15, Geocoding, Reverse Geocoding 제공 |
| Directions 5                 | 자동차 경로, 경유지 최대 5 개, 실시간 교통 반영                                |
| 한국관광공사 국문 관광정보               | 지역·위치·키워드·상세·이미지 등 제공                                        |
| Spring AI ChatClient         | 동기·스트리밍 모델 호출 지원                                             |
| Spring AI Structured Output  | Java 객체 형태로 구조화된 결과 변환                                       |

### 최종 전달 기준

두 백엔드는 Postman Collection 과 API.md 를, 프론트는 화면별 API 연결표를 제출한다. 요청·응답 DTO 가 달라지면 코드를 먼저 수정하지 말고 공통 API 계약을 갱신한 뒤 세 담당자가 확인한다.